



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт №3

Кафедра №301

Курсовая работа

по дисциплине

«Программирование и основы алгоритмизации»

на тему:

«Построение полинома Жегалкина методом суммирования»

Выполнила: студент гр. МЗО-101БВ-25

_____ К.В. Андреева

Принял: доцент кафедры 301

_____ С.С. Кананадзе

Москва – 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ	4
1.1 Полином Жегалкина	4
1.2 Построение полинома Жегалкина методом суммирования	5
2 РЕАЛИЗАЦИЯ ПРОГРАММЫ.....	7
2.1 Текстуальная запись алгоритма.....	7
2.2 Листинг программы	7
2.3 Взаимосвязь модулей программы	11
2.4 Парсер логических выражений.....	11
2.5 Анализ сложности алгоритма	13
2.6 Тестирование программы.....	14
2.7 Пример работы программы.....	16
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ВВЕДЕНИЕ

Традиционные методы построения полинома Жегалкина (например, метод неопределённых коэффициентов) требуют решения системы линейных уравнений, что трудоёмко при ручном счёте и не всегда удобно для автоматизации. Метод суммирования (треугольник Паскаля) менее известен, но алгоритмически прост и эффективен для программной реализации, однако они, доступные для учебных и практических целей, встречаются редко.

Цель исследования: Разработать программу, реализующую построение полинома Жегалкина для булевой функции от N переменных на основе метода суммирования (треугольной таблицы коэффициентов), и провести её тестирование на различных примерах.

Задачи:

- Изучить теоретические основы: полином Жегалкина, построение суммированием.
- Спроектировать структуру программы, реализовать алгоритм на выбранном языке программирования, протестировать.
- Проанализировать полученные результаты.

Методы исследования:

- Сбор и обработка информации из открытых научно-технических источников.
- Анализ научной литературы, учебников и пособий по исследуемой проблеме.

Объект исследования – процесс построения полинома Жегалкина для булевых функций методом суммирования.

Предмет исследования – алгоритмическая и программная реализация метода суммирования (треугольника Паскаля по модулю 2) для автоматического получения полинома Жегалкина.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

1.1 Полином Жегалкина

Полином Жегалкина (или многочлен Жегалкина) — это способ представления любой булевой функции в виде суммы по модулю 2 (XOR) произведений её аргументов.

Альтернативное название: алгебраическая нормальная форма (АНФ).

Представляет собой сумму попарно различных элементарных конъюнкций (произведений переменных без отрицаний), где каждая переменная в произведении встречается не более одного раза.

Общий вид для функции от n переменных:

$$P(x_1, \dots, x_n) = a_0 \oplus a_1 x_1 \oplus a_2 x_2 \oplus \dots \oplus a_n x_n \oplus a_{12} x_1 x_2 \oplus \dots \oplus a_{1\dots n} x_1 \dots x_n$$

где коэффициенты a_i принимают значения 0 или 1.

Основные свойства:

- **Единственность:** Каждая булева функция может быть представлена полиномом Жегалкина единственным образом (с точностью до порядка слагаемых).
- **Отсутствие инверсий и дизъюнкций:** В полиноме используются только операции конъюнкции (умножение) и сложения по модулю 2 (XOR). Отрицание заменяется на сложение по модулю 2 с единицей.
- **Линейность:** Функция является линейной тогда и только тогда, когда в её полиноме Жегалкина отсутствуют произведения двух и более переменных (содержит только константы и переменные в первой степени).
- **Количество:** Общее число различных полиномов Жегалкина от n переменных равно 2^{2^n} , что совпадает с общим количеством булевых функций от n переменных.

Методы построения:

1. **Метод эквивалентных преобразований:** Замена всех операций (И, ИЛИ, НЕ) на XOR и конъюнкцию с последующим раскрытием скобок.
2. **Метод неопределенных коэффициентов:** Составление системы уравнений на основе таблицы истинности для нахождения коэффициентов a_i .

3. Преобразование Мёбиуса: Линейное отображение таблицы истинности в коэффициенты полинома.

4. Метод суммирования, он же метод треугольника (Паскаля): Позволяет быстро найти коэффициенты, используя значения функции из таблицы истинности.

1.2 Построение полинома Жегалкина методом суммирования

Метод суммирования позволяет преобразовать таблицу истинности в полином Жегалкина путём построения вспомогательной треугольной таблицы (рисунок 1) в соответствии со следующими правилами:

1. Строится полная таблица истинности, в которой строки идут в порядке возрастания двоичных кодов от 000...00 до 111...11.

2. Строится вспомогательная треугольная таблица, в которой первый столбец совпадает со столбцом значений функции в таблице истинности.

3. Ячейка в каждом последующем столбце получается путём сложения по модулю 2 двух ячеек предыдущего столбца — стоящей в той же строке и строкой ниже.

4. Столбцы вспомогательной таблицы нумеруются двоичными кодами в том же порядке, что и строки таблицы истинности.

	000	001	010	011	100	101	110	111
<i>A</i>	1	<i>C</i>	<i>B</i>	<i>BC</i>	<i>A</i>	<i>AC</i>	<i>AB</i>	<i>ABC</i>
<i>B</i>	0	0	1	1	0	0	1	1
<i>C</i>	0	0	1	0	1	1	0	0
<i>P</i>	1	0	0	1	0	0	1	1

1	0	0	0	0	0	0	0
1	1	0	0	0	1	1	
0	1	0	1	1			
1	1	1	0				
0	0	1					
0	1						
1							

$$P = 1 \oplus C \oplus AB$$

Рисунок 1 Полином Жегалкина методом суммирования

5. Каждому двоичному коду ставится в соответствие один из членов полинома Жегалкина в зависимости от позиций кода, в которых стоят единицы. Например, ячейке 111 соответствует член ABC , ячейке 101 — член AC , ячейке 010 — член B , ячейке 000 — член 1 и т.д.

6. Если в верхней строке какого-либо столбца стоит единица, то соответствующий член присутствует в полиноме Жегалкина.

Удобен для ручного счета для небольшого числа переменных, прост в вычислениях, легко программируется. Метод широко используется в дискретной математике и теории булевых функций.

2 РЕАЛИЗАЦИЯ ПРОГРАММЫ

2.1 Текстуальная запись алгоритма

1. Начало алгоритма.
2. Ввод логического выражения.
3. Если выражение содержит только константы (нет переменных), то вывод на консоль сообщения об ошибке и переход к п. 7. Иначе — переход к п. 3.
4. Выполнение синтаксического разбора выражения и построение дерева вычислений (AST).
5. Генерация всех возможных комбинаций значений для найденных переменных и вычисление таблицы истинности, обходя построенное дерево.
6. Построение треугольной таблицы, вычисляя XOR соседних элементов вектора результатов, чтобы получить коэффициенты полинома Жегалкина.
7. Формирование строки полинома Жегалкина из конъюнкций с ненулевыми коэффициентами и вывод результата на консоль.
8. Конец алгоритма.

2.2 Листинг программы

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <memory>
5  #include <stdexcept>
6  #include <cctype>
7  #include <algorithm>
8  #include <set>
9  #include <windows.h>
10 using namespace std;
11
12 // ДАЛЬШЕ - узлы дерева, задают логику вычислений
13 struct ExprNode { // values — текущ знач переменных, varOrder — упорядоч список имён переменных
14     virtual bool eval(const vector<bool>& values, const vector<char>& varOrder) const = 0;
15     virtual ~ExprNode() = default;
16 }; // базовый узел: вирт функц, которую переопред наследники + авто деструктор для очистки
17
18 struct VarNode : ExprNode {
19     char name;
20     VarNode(char c) : name(c) {} // конструктор, создание объекта
21
22     bool eval(const vector<bool>& values, const vector<char>& varOrder) const override {
23         // поиск индекса переменной в varOrder
24         auto it = find(varOrder.begin(), varOrder.end(), name);
25         int idx = distance(varOrder.begin(), it);
26         return values[idx];
27     }
28 }; // узел переменной
29
30 struct NotNode : ExprNode {
31     unique_ptr<ExprNode> child;
32     NotNode(unique_ptr<ExprNode> c) : child(move(c)) {}
33     bool eval(const vector<bool>& values, const vector<char>& varOrder) const override {
34         return !child->eval(values, varOrder);
35     } // полиморфизм eval - единый, работает с разными типами узлов (остальные eval)
36 }; // узел не
37
```

```

38 struct AndNode : ExprNode {
39     unique_ptr<ExprNode> left, right;
40     AndNode(unique_ptr<ExprNode> l, unique_ptr<ExprNode> r)
41         : left(move(l)), right(move(r)) {}
42     bool eval(const vector<bool>& values, const vector<char>& varOrder) const override {
43         return left->eval(values, varOrder) && right->eval(values, varOrder);
44     }
45 }; // узел и
46
47 struct OrNode : ExprNode {
48     unique_ptr<ExprNode> left, right;
49     OrNode(unique_ptr<ExprNode> l, unique_ptr<ExprNode> r)
50         : left(move(l)), right(move(r)) {}
51     bool eval(const vector<bool>& values, const vector<char>& varOrder) const override {
52         return left->eval(values, varOrder) || right->eval(values, varOrder);
53     }
54 }; // узел или
55
56 // ДАЛЬШЕ - таблица истинности, объединяет узлы и парсер
57 class TruthTable {
58 public:
59     explicit TruthTable(const string& expr) {
60         set<char> varSet; // динамический массив для сбора переменных
61         int pos = 0;
62         root = parse_expression(expr, pos, varSet);
63         skip_spaces(expr, pos);
64         if (pos != expr.size()) {
65             throw runtime_error("Лишние символы после выражения");
66         }
67         vars.assign(varSet.begin(), varSet.end()); // упорядоченный список переменных
68         int n = vars.size();
69
70         table.resize(1 << n);
71         for (int i = 0; i < (1 << n); ++i) {
72             vector<bool> values(n);
73             for (int j = 0; j < n; ++j) {
74                 values[j] = (i >> (n - 1 - j)) & 1;
75             }
76             table[i] = root->eval(values, vars);
77         } // заполнение таблицы истинности
78     } // запуск парсера, сбор имен переменных, генер комбинаций, заполн табл
79
80     const vector<bool>& get_table() const { return table; }
81     const vector<char>& get_variables() const { return vars; }
82     // доступ к приватным таблице и именам переменных
83
84 private:
85     vector<char> vars; // имена переменных
86     vector<bool> table; // таблица истинности
87     unique_ptr<ExprNode> root; // AST (дерево)
88
89     // ДАЛЬШЕ - парсер, строит дерево из строки
90     void skip_spaces(const string& s, int& i) {
91         while (i < s.size() && isspace((unsigned char)s[i])) ++i;
92     } // скип пробелов в строке
93
94     unique_ptr<ExprNode> parse_expression(const string& s, int& i, set<char>& varSet) {
95         auto left = parse_term(s, i, varSet);
96         skip_spaces(s, i);
97         while (i < s.size() && s[i] == '+') {
98             ++i;
99             auto right = parse_term(s, i, varSet);
100             left = make_unique<OrNode>(move(left), move(right));
101             skip_spaces(s, i);
102         } // кидает в след функцию - или
103         return left;
104     } // основа дерева, получает данные, закидывает в след функцию

```



```

105
106 unique_ptr<ExprNode> parse_term(const string& s, int& i, set<char>& varSet) {
107     auto left = parse_factor(s, i, varSet);
108     skip_spaces(s, i);
109     while (i < s.size() &&
110         (s[i] == '*' || isalpha(s[i]) || s[i] == '(' || s[i] == '!')) {
111         if (s[i] == '*') ++i;
112         auto right = parse_factor(s, i, varSet);
113         left = make_unique<AndNode>(move(left), move(right));
114         skip_spaces(s, i);
115     } // кидает в след функцию - и
116     return left;
117 } // проверяет продолжается ли произведение, закид в след функцию
118
119 unique_ptr<ExprNode> parse_factor(const string& s, int& i, set<char>& varSet) {
120     skip_spaces(s, i);
121     if (i >= s.size()) throw runtime_error("Неожиданный конец выражения");
122
123     if (s[i] == '!') {
124         ++i;
125         return make_unique<NotNode>(parse_factor(s, i, varSet));
126     } // кидает в след функцию - не
127     else if (isalpha(s[i])) {
128         char var = s[i];
129         ++i;
130         varSet.insert(var);
131         return make_unique<VarNode>(var);
132     } // запоминает новую переменную
133     else if (s[i] == '(') {
134         ++i;
135         auto inside = parse_expression(s, i, varSet);
136         skip_spaces(s, i);
137         if (i >= s.size() || s[i] != ')')
138             throw runtime_error("Ожидалась ')'");
139         ++i;
140         return inside;
141     } // ошибка не закрыта скобка
142     else {
143         throw runtime_error(string("Неожиданный символ: ") + s[i]);
144     }
145 } // добавляет буквы в словарь, подсчитывает слагаемые
146 };
147
148
149 string tabl(const TruthTable& table)
150 {
151     const auto& vars = table.get_variables();
152     const auto& data = table.get_table();
153     int n = vars.size();
154     int rows = 1<n;
155
156     vector<int> current;
157     for (bool b: data) current.push_back(b ? 1:0);
158
159     vector<string> conjName(rows); // строковые имена переменных
160     for (int i=0; i<rows; ++i) {
161         if (i==0) { conjName[i]="1";
162         } else {
163             string conj;
164             for (int j=0; j<n; ++j) {
165                 if (i & (1<(n-1-j))) {
166                     conj+=vars[j];
167                 }
168             }
169             conjName[i]=conj;
170         }
171     }
172
173     cout << "\nТреугольная таблица\n";
174     vector<int> coefficients;

```

```

175     int step=0;
176     while (current.size()>1) {
177         cout << conjName[step]<< " : ";
178         for (int val : current) {
179             cout << val << ' ';
180         }
181         cout << "\n";
182         coefficients.push_back(current[0]);
183         vector<int> next;
184         for (size_t i=0; i+1 < current.size(); ++i) {
185             next.push_back(current[i] ^ current[i+1]);
186         }
187         current = move(next);
188         ++step;
189     }
190     cout << conjName[step] << " : " << current[0] << '\n';
191     coefficients.push_back(current[0]);
192
193     string polynom;
194     for (int i=0; i<rows; ++i) {
195         if (coefficients[i]) {
196             if (!polynom.empty()) polynom += " ^ ";
197             polynom += (i==0) ? "1" : conjName[i];
198         }
199     }
200     if (polynom.empty()) polynom="0";
201     return polynom;
202 }
203
204 int main()
205 {
206     SetConsoleOutputCP(CP_UTF8);
207     SetConsoleCP(CP_UTF8);
208
209     unique_ptr<TruthTable> tablePtr; // таблица знач функции
210     string expr;
211     string polynom = "";
212     char c;

```

```

211
212     while (true) {
213         cout << "\n1. Ввести функцию по которой нужно построить полином.";
214         cout << "\n2. Вывести таблицу построения полинома.";
215         cout << "\n3. Вывести построенный полином Жегалкина.";
216         cout << "\n4. Завершить работу.";
217         cout << "\n> ";
218         cin >> c;
219         cin.ignore();
220
221         switch(c) {
222             case '1': {
223                 cout << "Введите функцию (переменные - большие латинские буквы, знаки - '+', '*', '!): \n";
224                 getline(cin, expr);
225                 try {
226                     tablePtr=make_unique<TruthTable>(expr);
227                 } catch (const exception& e) {
228                     cerr << "Ошибка: " << e.what() << endl;
229                     tablePtr.reset();
230                 }
231                 break;
232             }
233             case '2': {
234                 if (tablePtr) {
235                     polynom=tabl(*tablePtr);
236                 } else {
237                     cout << "Сначала введите функцию (пункт 1).\n";
238                 }
239                 break;
240             }
241             case '3': {
242                 if (polynom!="") {
243                     cout << "Полином Жегалкина от функции " << expr << ":\n";
244                     cout << polynom << "\n";
245                 } else {
246                     cout << "Сначала постройте таблицу полинома (пункт 2).\n";
247                 }
248                 break;
249             }
250         }
251     }

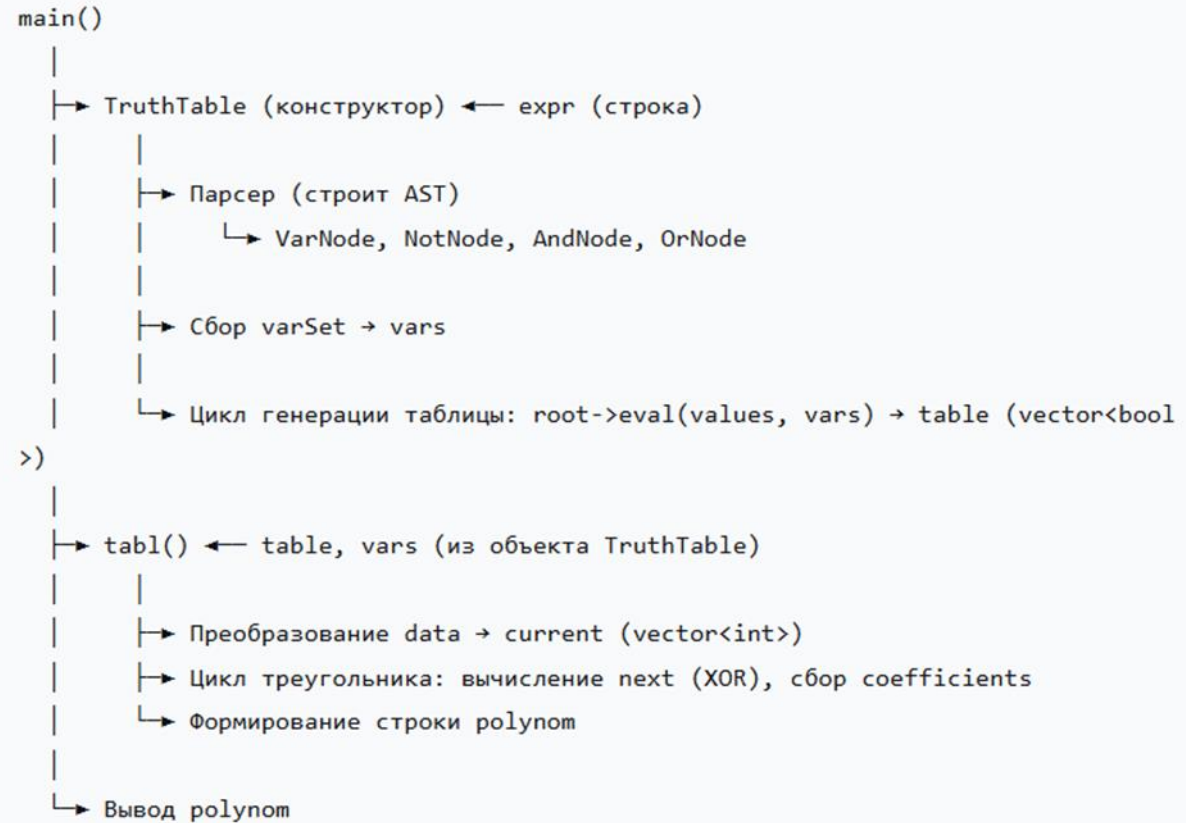
```

```

248     }
249     case '4': return 0;
250     default: cout << "Вводите цифры от 1 до 4\n";
251 }
252 }
253 }

```

2.3 Взаимосвязь модулей программы



2.4 Парсер логических выражений

В программе реализован нисходящий рекурсивный парсер, который строит абстрактное синтаксическое дерево (AST) для последующего вычисления логических выражений. Такой подход обеспечивает гибкость, расширяемость и надёжность при работе с произвольными формулами.

Техническая реализация:

1. Иерархия узлов (AST):

- Базовый класс ExprNode — абстрактный интерфейс с виртуальным методом eval, реализующим вычисление значения узла. Все конкретные узлы наследуют этот интерфейс.
- VarNode — узел переменной. Хранит имя переменной и реализует поиск её значения в векторе значений по упорядоченному списку переменных.

- `NotNode` — узел отрицания. Хранит указатель на единственный дочерний узел (через `unique_ptr`) и инвертирует результат его вычисления.
- `AndNode` и `OrNode` — бинарные узлы для операций конъюнкции и дизъюнкции. Каждый хранит два дочерних узла и реализует соответствующую логическую операцию.

2. Управление памятью:

Используется автоматическое освобождение памяти при удалении дерева, что гарантирует отсутствие утечек и исключений при копировании.

3. Сбор переменных:

В процессе парсинга все встреченные переменные добавляются в `std::set<char>`, что обеспечивает уникальность и автоматическую сортировку. После парсинга этот набор преобразуется в вектор, задающий порядок переменных для таблицы истинности.

4. Алгоритм парсинга (рекурсивный спуск):

Парсер реализован как набор взаимно-рекурсивных функций: `parse_expression` (дизъюнкция), `parse_term` (конъюнкция), `parse_factor` (атомарные выражения). Каждая функция возвращает `unique_ptr<ExprNode>`, что позволяет строить дерево «на лету». Пробелы пропускаются отдельной функцией `skip_spaces`.

5. Обработка ошибок:

При обнаружении синтаксических ошибок (неожиданный конец строки, лишние символы, незакрытые скобки) выбрасывается исключение `std::runtime_error` с информативным сообщением.

6. Эффективность:

Использование ссылок (`const vector<bool>&`, `const vector<char>&`) в методе `eval` минимизирует копирование данных. Поиск индекса переменной в векторе осуществляется стандартным алгоритмом `std::find`.

Преимущества реализации: Чёткое разделение логики парсинга, хранения данных и вычислений и гибкость: легко добавить новые типы узлов (например, для импликации или эквивалентности).

Такой подход позволяет парсеру корректно обрабатывать сложные выражения с произвольной вложенностью и гарантирует корректную работу всей системы вычисления полинома Жегалкина.

2.5 Анализ сложности алгоритма

Анализ сложности алгоритма проводится отдельно для двух основных этапов работы программы: построения таблицы истинности и вычисления полинома Жегалкина.

1. Анализ этапа построения таблицы истинности.

Этот этап включает в себя синтаксический анализ (парсинг) и вычисление значения выражения для всех возможных комбинаций переменных.

Сложность по времени (Time Complexity)

1. Парсинг выражения: Парсер использует алгоритм рекурсивного спуска. Время работы парсера линейно зависит от длины входной строки L . Следовательно, сложность этого шага — $O(L)$.

2. Генерация и вычисление таблицы: n — количество уникальных переменных в выражении, количество строк в таблице истинности равно 2^n . Для каждой из 2^n строк необходимо вычислить значение выражения. Это (обход дерева AST) требует времени, пропорционального размеру дерева, который, в свою очередь, пропорционален длине выражения L , т.е. $O(L * 2^n)$.

Общая временная сложность для этого этапа: $O(L) + O(L * 2^n) = O(L * 2^n)$.

Сложность по памяти (Space Complexity)

1. Память для AST (дерева): Дерево синтаксического анализа хранит структуру выражения. Его размер пропорционален длине входной строки L . Сложность — $O(L)$.

2. Память для данных: вектор `vars` для хранения имён переменных: $O(n)$ и вектор `table` для хранения результатов вычислений: $O(2^n)$.

Общая сложность по памяти для этапа: $O(L) + O(n) + O(2^n) = O(L + 2^n)$.

2. Анализ этапа вычисления полинома Жегалкина.

Этот этап выполняется в функции `tabl` и работает с уже построенной таблицей истинности.

Сложность по времени (Time Complexity)

1. Генерация имён конъюнкций (`conjName`): Внешний цикл выполняется 2^n раз. Внутренний цикл для каждой строки выполняется n раз. Итого — $O(n \cdot 2^n)$.

2. Построение треугольной таблицы: Внешний цикл (по `step`) выполняется $n+1$ раз. Внутренние циклы обрабатывают массивы, размер которых на каждой итерации уменьшается вдвое: 2^n , 2^{n-1} и тд. Суммарное количество операций: $2^n + 2^{n-1} + \dots + 1 = 2^{n+1} - 1$. Итого сложность — $O(2^{n+1})$.

3. Формирование итоговой строки полинома: Цикл по вектору коэффициентов размера 2^n . Сложность шага — $O(2^n)$.

Общая временная сложность для этого этапа: $O(n \cdot 2^n) + O(2^{n+1}) + O(2^n) = O(n \cdot 2^n)$.

Сложность по памяти (Space Complexity)

1. Вектор `current` и `next`: В сумме занимают не более $O(2^n)$ памяти.
2. Вектор `conjName`: Хранит 2^n строк, каждая из которых может иметь длину до n . Сложность — $O(n \cdot 2^n)$.
3. Вектор `coefficients`: Хранит 2^n целых чисел. Сложность — $O(2^n)$.
4. Строка `polynom`: В худшем случае может содержать все конъюнкции, что также даёт сложность $O(n \cdot 2^n)$.

Общая сложность по памяти для этого этапа: $O(n \cdot 2^n)$.

2.6 Тестирование программы

Тестирование программы проводилось с целью проверки корректности реализации алгоритма построения полинома Жегалкина методом суммирования, а также для оценки работоспособности на различных типах входных данных. В рамках тестирования были выделены следующие направления:

Функциональное тестирование — проверка правильности работы программы на известных примерах булевых функций, для которых полином Жегалкина можно получить вручную.

Граничное тестирование — анализ работы программы при минимальном и максимальном количестве переменных, а также при обработке вырожденных случаев (например, константные функции, отсутствие переменных).

Тестирование на ошибочных и некорректных входных данных — проверка реакции программы на синтаксические ошибки, лишние символы, некорректные имена переменных и другие аномалии.

Примеры тестовых случаев (вывод):

1. Функция двух переменных: Вход: $A \oplus B$.

Полином Жегалкина от функции $(A+B) \cdot (A \cdot B)$:
 $B \wedge A$

2. Функция трёх переменных: Вход: $(A \wedge B) \vee C$.

Полином Жегалкина от функции $AB + C$:
 $C \wedge AB \wedge ABC$

3. Константная функция: Вход: 1.

Ошибка: Неожиданный символ: 1

4. Функция с одной переменной: Вход: A.

Полином Жегалкина от функции A:
A

5. Некорректный ввод (синтаксическая ошибка): Вход: $A \&\& B$.

$A\&\&B$

Ошибка: Лишние символы после выражения

Результаты тестирования:

Все основные функциональные тесты подтвердили корректность работы программы: для каждого из рассмотренных примеров полученный полином Жегалкина совпал с ожидаемым результатом. Граничные случаи обрабатываются корректно, программа устойчива к некорректным и ошибочным входным данным, выдавая понятные сообщения об ошибках.

2.7 Пример работы программы

1. Ввести функцию по которой нужно построить полином.
 2. Вывести таблицу построения полинома.
 3. Вывести построенный полином Жегалкина.
 4. Завершить работу.
- > 1

Введите функцию (переменные - большие латинские буквы, знаки - '+', '*', '!'):
AB+AC(A+B)

Треугольная таблица

1 : 0 0 0 0 0 1 1 1

C : 0 0 0 0 1 0 0

B : 0 0 0 1 1 0

BC : 0 0 1 0 1

A : 0 1 1 1

AC : 1 0 0

AB : 1 0

ABC : 1

Полином Жегалкина от функции AB+AC(A+B):

AC ^ AB ^ ABC

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были изучены теоретические основы построения полинома Жегалкина, особое внимание уделено методу суммирования (треугольной таблицы коэффициентов), который отличается простотой, наглядностью и высокой эффективностью при автоматизации.

На основе проведённого исследования был разработан и реализован программный модуль, позволяющий по заданной булевой функции автоматически строить её полином Жегалкина. Тестирование программы подтвердило её корректность.

Таким образом, поставленная цель достигнута: программа для построения полинома Жегалкина методом суммирования успешно реализована, протестирована и может быть использована как в учебных, так и в практических задачах дискретной математики и теории булевых функций. Полученный опыт может быть полезен дальнейшего развития.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Давыдов, В. Г. *Программирование и основы алгоритмизации: учебное пособие для вузов*. — Москва: Высшая школа, 2003. — 448 с. — ISBN 5-06-004432-7.
2. Полином Жегалкина [Электронный ресурс]. — URL: <https://system-of-bf.azurewebsites.net/Content/Pages/Polynom.aspx> (дата обращения: 14.05.2026).
3. Полином Жегалкина [Электронный ресурс] // Википедия. — URL: https://ru.wikipedia.org/wiki/Полином_Жегалкина (дата обращения: 14.05.2026).
4. Что нам стоит полином Жегалкина построить... [Электронный ресурс] // Habr. — URL: <https://habr.com/ru/articles/275527/> (дата обращения: 14.05.2026).